

Consolidated Alarm Solution (CAS): Master Engineering Specifications for Edge Application and Flink Alarm Lifecycle Management

The architectural mandate presented herein dictates the engineering of a Consolidated Alarm Solution (CAS) designed to aggregate, normalise, and manage the full lifecycle of alarms across 74 distributed sites. Operating within a highly heterogeneous, multi-vendor Distributed Control System (DCS) landscape, this architecture establishes an authoritative, real-time, and historical alarm presentation layer at the Integrated Operations Centre (IOC).¹ The design unequivocally rejects legacy wide-area DCOM communication and third-party OPC tunnellers, establishing instead a secure, edge-to-broker telemetry pattern utilising Apache Kafka for robust store-and-forward buffering, and Apache Flink for stateful, exactly-once alarm lifecycle processing. The following document constitutes the exhaustive, implementation-ready specifications required to construct both the local Edge Application and the enterprise Flink Alarm-Lifecycle State-Management engine in strict alignment with ISA-18.2 and EEMUA 191 standards.

DELIVERABLE A — EDGE APPLICATION SPECIFICATION

1. Purpose, Deployment Context, and Boundary Conditions

The Edge Application serves as the foundational data acquisition and normalisation tier, deployed directly within the Process Control Domain (PCD) of each of the 74 operational sites. Its primary directive is to interface securely with native DCS OPC Alarms & Events (A&E) servers, translate proprietary, vendor-specific COM structures into a strictly versioned canonical schema, and buffer this data locally to survive wide-area network partitions.¹ To satisfy the absolute prohibition of remote DCOM and external OPC tunnellers, the deployment architecture mandates local COM instantiation.¹ This necessitates one of two deployment topologies. The primary option is a co-resident deployment, where a lightweight edge agent executes directly upon the OEM OPC server host. This approach eliminates network-based DCOM entirely, mitigating severe vulnerabilities associated with RPC endpoint mappers (e.g., CVE-2021-26414) and aligning with modern hardening protocols.² Where OEM warranty stipulations or stringent PCD policies explicitly prohibit the installation of third-party executables on vendor-supplied nodes, the secondary option dictates the provisioning of an adjacent virtual machine located on the identical local subnet. This adjacent deployment restricts DCOM traffic to a strictly controlled, local-subnet boundary, managed via precise Windows Firewall rules ensuring minimal attack surface exposure.¹

ASSUMPTION: It is assumed that the physical or virtual hardware hosting the OPC A&E servers possesses sufficient residual compute and storage capacity to accommodate the edge agent's memory footprint and disk buffer. **OPEN ITEM:** Formal approval for co-resident deployment versus adjacent VM provisioning must be secured per OEM vendor from the operational technology (OT) cybersecurity and maintenance authorities.¹ **OPEN ITEM:** The permissibility of deploying the edge agent as an OCI-compliant container (e.g., Docker) within the PCD remains unconfirmed; the architecture must support both containerised and native Windows Service execution.¹

2. OEM Connector Abstraction

The edge agent is engineered around a pluggable, driver-based connector abstraction. The core application runtime governs the Kafka producer lifecycle, local write-ahead logging, and thread pooling, while dynamically loaded modules implement the specific data extraction protocols required by each DCS vendor.

The primary driver leverages the OPC Foundation's IOPCEventServer and IOPCEventSubscriptionMgt Custom COM interfaces to interact with standard A&E servers.¹ To successfully instantiate the COM objects, the connector relies on specific Programmatic Identifiers (ProgIDs) associated with each OEM. For instance, the Emerson DeltaV system exposes DeltaV.OPCEventServer.1³, while Yokogawa's Exaopc platform exposes Yokogawa.ExaopcAESCSSOE or Yokogawa.ExaopcAECS1.⁴ ABB 800xA systems utilise ABB.OPCEventServer (historically known as AfwEventOPCServer).⁶ The application must programmatically resolve these ProgIDs to their respective Class IDs (CLSIDs) without relying on remote OPCEnum services, which frequently fail across firewalled subnets.⁸

Foxboro I/A Series and Foxboro Evo control systems present a deliberate architectural exception. These systems do not expose a native, clean OPC A&E interface comparable to their peers.¹ Consequently, the Foxboro connector is implemented as a specialised JDBC/ODBC SQL polling driver. This driver executes scheduled queries against the AIM*Historian or embedded SQL alarm tables to extract process alarms and events.¹⁰

ASSUMPTION: The Foxboro SQL database schema allows for sequential, indexed reads of the alarm tables without applying table-level locks that could disrupt the primary DCS operation.

OPEN ITEM: The exact Foxboro SQL database schema (e.g., table columns, flog structures, alarm acknowledgment status flags) must be documented to finalise the query logic and confirm that a ≤5-second synchronisation target is technically feasible without inducing excessive database load.¹ **OPEN ITEM:** While the current architecture relies on OPC Classic, future expansion necessitates an OPC UA Alarms & Conditions (A&C) driver. The application must support a modular substitution of the COM driver with a UA client.¹

3. Connection & Subscription Management

To safeguard the performance of the underlying DCS, the edge application is strictly limited to establishing a single event subscription per OPC A&E server.¹ The architecture fundamentally rejects the practice of utilizing per-tag OPC Data Access (DA) polling to infer alarm states,

relying exclusively on event-driven push notifications.¹

Upon successful COM instantiation and authentication, the client invokes `IOPCEventServer::CreateEventSubscription` with the `bActive` parameter set to `TRUE`, establishing the asynchronous callback pipeline.¹ To minimise edge-to-broker bandwidth and reduce local processing overhead, the application pushes filtering down to the OPC server using the `IOPCEventSubscriptionMgt::SetFilter` method.¹ The filter is configured to request only `OPC_CONDITION_EVENT` and `OPC_SIMPLE_EVENT` types, deliberately excluding high-volume, low-value tracking events unless specific operator audit trails are requested by the IOC.¹ A critical mechanism in the subscription lifecycle is the explicit invocation of the `Refresh()` method. The OPC A&E specification dictates that simply creating an active subscription does not inherently trigger an initial state dump.¹ Therefore, upon every successful initial connection or subsequent reconnection following a network or service disruption, the edge agent must issue `IOPCEventSubscriptionMgt::Refresh`.¹ The OPC server responds asynchronously via the `IOPCEventSink::OnEvent` callback, passing a batch of notifications with the `bRefresh` flag set to `TRUE`, concluding with a batch where `bLastRefresh` is `TRUE`.¹ This sequence is vital for state reconciliation, ensuring the edge buffer is seeded with an accurate snapshot of all currently active or unacknowledged conditions.¹

To monitor connection health, the agent invokes `IOPCEventServer::GetStatus` at a configurable 30-second interval.¹ Should this call return `OPCAE_STATUS_FAILED`, `OPCAE_STATUS_COMM_FAULT`, or yield an RPC timeout, the agent will gracefully release all COM references, tear down the apartment threads, and initiate an exponential backoff reconnection sequence.¹

4. Event Ingestion & Canonical Schema

As vendor-specific `ONEVENTSTRUCT` payloads arrive via the COM callback¹, they are immediately deserialised and transformed into a unified, strongly typed canonical schema. This schema acts as the universal language for the entire enterprise, insulating the central IOC Flink engine from the idiosyncrasies of individual OEM structures. The canonical record is serialised using Apache Avro (or Protocol Buffers) and registered within an Edge Schema Registry to enforce strict schema evolution rules (e.g., backward compatibility) across all 74 sites. The canonical schema, defined as `CasAlarmEvent`, incorporates the following fields:

Field Name	Data Type	Source Mapping / Description
eventId	String (UUID)	A globally unique identifier generated at the edge upon ingestion.
siteId	String	Static configuration

		parameter identifying the physical facility.
oemType	Enum	Determines specific normalisation logic (HONEYWELL, EMERSON, YOKOGAWA, FOXBORO, ABB, GE, MATRIKON).
serverId	String	Identifies the specific redundant node or OPC server instance.
source	String	Maps directly to the OPC szSource attribute (e.g., the tag or module path). ¹
conditionName	String	Maps to the OPC szConditionName (e.g., LevelAlarm, Deviation). ¹
subCondition	String	Maps to the OPC szSubconditionName (e.g., HI, HIHI, LO). Null for single-state events. ¹
eventType	Enum	Mapped from dwEventType (CONDITION, SIMPLE, TRACKING). ¹
severityRaw	Int	The native priority/severity value provided by the DCS. ¹
severityNormalised	Int	The mapped severity value standardised to the 1-1000 OPC range. ¹
priorityNormalised	Enum	The CAS enterprise priority tier (CRITICAL, HIGH, MEDIUM, LOW, JOURNAL).

activeTime	Long (Epoch ms)	Maps to ftActiveTime. This is the true controller SOE timestamp and the stable correlation anchor for the alarm instance. ¹
eventTime	Long (Epoch ms)	Maps to ftTime. The time of the specific notification or transition. ¹
changeMask	Int	Maps to wChangeMask. Crucial for deduplication; indicates which specific attributes changed without a state transition. ¹
newState	Record	A nested record containing three booleans (enabled, active, acked), unpacked via bitwise operations on wNewState. ¹
ackRequired	Bool	Maps to bAckRequired, dictating if the transition necessitates operator intervention. ¹
quality	Int	Maps to wQuality, indicating the health of the underlying tag (e.g., OPC_QUALITY_BAD). ¹
message	String	Maps to szMessage, providing the operator-readable description. ¹
area	String	Extracted from vendor-specific attributes or parsed from the source hierarchy.

cookieRef	Int	Maps to dwCookie. A highly volatile handle required strictly for write-back acknowledgements. ¹
ingestTime	Long (Epoch ms)	The timestamp recorded by the edge agent upon receipt of the event.
timestampFidelity	Enum	Flags the precision of the activeTime (CONTROLLER_SOE, SERVER_RECEIVE, EDGE_INGEST).

5. Normalisation at the Edge

The disparity between OEM DCS priority models requires a robust normalisation engine at the edge. The OPC A&E specification defines a severity range of 1 to 1000¹; however, individual vendors implement this erratically. The CAS architecture enforces a unified five-tier priority model. The edge agent executes the following normalisation matrices based on the oemType configuration:

OEM Platform	Native Priority Model	OPC Severity Internal Mapping	Target CAS Priority	Timestamp Fidelity
Honeywell Experion PKS	Journal, Low, High, Urgent ¹³	Journal(1), Low(150), High(500), Urgent(1000) ¹⁴	JOURNAL, LOW, HIGH, CRITICAL	CONTROLLER_SOE (Converted from internal UTC bias) ¹⁵
Emerson DeltaV	Advisory, Warning, Critical, Log ¹⁶	Advisory(250), Warning(600), Critical(1000) ¹⁸	LOW, MEDIUM, CRITICAL, JOURNAL	CONTROLLER_SOE
Yokogawa CENTUM VP	Priority 3 (Low), Priority 2 (Medium),	Ranges mapped 200-900.	LOW, MEDIUM, HIGH, CRITICAL	CONTROLLER_SOE ⁴

	Priority 1 (High) ¹⁹	System Alarms mapped to 900. ⁴		
ABB 800xA / AC 800M	Low, Medium, High, Critical (Fatal) ²⁰	Low(1-250), Medium(251-500), High(501-750), Fatal(751-1000) ²⁰	LOW, MEDIUM, HIGH, CRITICAL	CONTROLLER_SOE
GE WorkstationST	Configurable per diagnostic block ²¹	Extracted from specific OPC_AE_Severity properties ²¹	Evaluated via dynamic ruleset	CONTROLLER_SOE
Foxboro I/A Series	Numerical 1-5 levels via SQL schema ¹¹	N/A (Bypasses OPC COM interfaces)	Mapped directly: 1=CRITICAL, 5=JOURNAL	EDGE_INGEST

OPEN ITEM: The Foxboro timestamp fidelity is currently assigned as EDGE_INGEST. Foxboro historians frequently record the server receive time rather than the true controller millisecond sequence of events.¹ PDO must confirm if true SOE is available in the SQL schema; otherwise, the ≤5-second synchronisation target becomes the de facto precision limit for Foxboro data.¹

6. Store-and-Forward & Backfill

The absolute requirement for zero alarm loss across network outages mandates a durable, high-throughput buffer at the edge.¹ The edge application integrates a lightweight, single-node Apache Kafka broker (or an API-compatible equivalent like Redpanda) to serve as the write-ahead log.

All normalised CasAlarmEvent records are published synchronously to the local topic cas.edge.telemetry.alarms.raw. The Kafka retention policy is strictly size-based, configured to retain data for a prolonged network partition. During normal operations, Kafka MirrorMaker 2 (or native Cluster Linking) asynchronously replicates the topic byte-for-byte to the IOC Kafka cluster across the DMZ over mutual TLS.¹

In the event of a wide-area network failure, the replication process stalls, but the edge application continues to ingest and append events to the local log without backpressure blocking the OPC COM thread. Upon network restoration, replication resumes automatically from the last committed offset. Crucially, the Kafka architecture ensures that backfilled events are transmitted in the exact sequential order they were generated, preserving the temporal integrity required for sequence-of-events analysis.

ASSUMPTION: Based on typical DCS alarm flood metrics, the sizing calculation assumes a worst-case peak alarm flood rate of 1,000 events per second per site. Assuming an average Avro payload of 500 bytes, generating 500 KB/second. To survive a 72-hour network partition, the edge broker requires a minimum of 130 GB of dedicated SSD storage solely for the telemetry buffer.¹ **OPEN ITEM:** PDO must provide definitive peak alarm flood rates and the required maximum outage duration to mathematically finalise the edge hardware storage sizing.¹

7. Acknowledgement / Command Handling (Write-Back)

The presentation layer requires the capability for operators to acknowledge or shelve alarms directly from the HTML5 banner.¹ This mandates a highly reliable write-back path from the IOC down to the originating edge OPC server.

A significant architectural challenge lies in the OPC A&E specification's handling of acknowledgements. The `IOPCEventServer::AckCondition()` method requires the client to supply the `szSource`, the `szConditionName`, the `ftActiveTime`, and a server-generated `dwCookie`.¹ While the source, condition, and active time form a stable, immutable identifier for the alarm instance, the Cookie is highly volatile. It is an ephemeral session handle; if the edge agent experiences a transient failure and reconnects, the subsequent `Refresh()` operation will issue entirely new Cookies for the identical active alarms.¹

To resolve this, the edge agent maintains a high-performance, in-memory Map Stateful Cache mapping the immutable tuple (Source, ConditionName, ActiveTime) to the currently active Cookie. When the IOC issues an operator command, it publishes an `AckCommand` record to the `cas.ioc.commands.outbound` topic, which is replicated down to the edge. The edge agent consumes the command, queries its internal state map using the tuple, retrieves the valid Cookie, and invokes the `COM AckCondition()` method.¹

Crucially, the architecture adheres to a strict "Confirmation-via-DCS" paradigm. The edge agent and the UI **never** optimistically finalise the acknowledged state upon button press.¹ The system waits for the DCS to process the acknowledgement and emit a new `OPC_CONDITION_EVENT` containing an updated `wNewState` where the `OPC_CONDITION_ACKED` bit is logically high.¹ This definitive event flows back up the forward path to update the enterprise Flink state and subsequently the UI, guaranteeing that the operator banner strictly reflects the ground truth of the DCS.

OPEN ITEM: The feasibility of this entire write-back mechanism hinges entirely on the network security architecture at the PCD/IOC boundary. If PDO enforces a hardware data-diode, inbound commands are physically impossible, and acknowledgement must remain restricted to local DCS consoles. If a bidirectional firewall is deployed, precise port rules are required.¹

8. Security

The edge tier operates under a zero-trust model tailored for OT environments.

- **Network Isolation:** Northbound communication from the edge broker to the IOC is strictly encrypted using TLS 1.3. Mutual TLS (mTLS) with x.509 certificates ensures that only cryptographically verified edge nodes can publish to the enterprise broker.

- **COM Hardening:** By enforcing local communication between the agent and the OPC server, the system negates the need to open port 135 and the dynamic RPC port range across firewalls. The edge application executes under a least-privilege Windows Service account, granted explicit Local Launch and Local Access permissions via the Component Services administrative tool (dcomcnfg).⁶
- **Host Security:** The agent is compiled to be application-whitelisting friendly (e.g., signed executables compliant with AppLocker or Trellix Application Control). It does not dynamically download executable code or execute arbitrary scripts.

9. Resource & Footprint

The edge application is engineered for minimal impact on aging DCS hardware.

- **Co-resident Envelope:** When deployed directly on an OEM OPC server, the application operates within a hard constraint of 1 vCPU and 1 GB of RAM to prevent resource starvation of the primary control system. The local Kafka buffer requires dedicated IOPS and should ideally be mounted on a separate logical partition with 50 GB of storage.
- **Adjacent VM Envelope:** When provisioned as a standalone edge gateway, the VM should be allocated 4 vCPUs, 8 GB of RAM, and 200 GB of SSD storage to handle the Java Virtual Machine (JVM) heap, OS page cache for Kafka, and the 72-hour store-and-forward buffer.

10. Configuration & Lifecycle

Configuration management relies on a centralised GitOps repository. Site-specific parameters, OEM connector profiles, and tag filter lists are maintained as declarative YAML/JSON files. Changes are pushed to edge nodes via an automated CI/CD pipeline or a dedicated edge orchestration platform.

The edge agent does not rely on static configuration files to determine which alarms to process. It asynchronously processes the event stream as it arrives. While the IOPCEventAreaBrowser interface¹ can be utilised to query the hierarchical plant namespace for diagnostic purposes, runtime discovery is fully dynamic. When a new alarm is configured in the DCS and triggers, the agent immediately captures the event, serialises it, and forwards it to the IOC without requiring edge-level reconfiguration.¹

11. Observability

The edge agent embeds a robust observability stack, exposing an HTTP endpoint formatted for Prometheus scraping (/metrics). Key performance indicators include:

- `opc_connection_status`: Binary gauge representing the COM connection state.
- `events_ingested_total`: Counter tracking volume from the DCS.
- `com_rpc_latency_ms`: Histogram detailing the execution time of OPC method calls.
- `local_buffer_size_bytes`: Gauge monitoring the Kafka disk footprint.
- `buffer_replication_lag`: Tracks the offset delta between the edge log and the IOC mirror.

12. Failure Modes & Test Plan

The system must be subjected to rigorous Factory Acceptance Testing (FAT).

- **COM Crash:** Terminate the OPC server process abruptly. The agent must detect the broken RPC channel, log a critical error, and enter an exponential backoff loop without memory leaking.¹
- **Network Partition (The S&F Test):** Physically sever the outbound network link. Generate 50,000 synthetic alarms in the DCS. Restore the link. Verification requires validating that exactly 50,000 alarms appear in the IOC Kafka cluster in exact chronological order, with no duplication.
- **Agent Termination:** Kill the edge agent process mid-stream. Upon restart, the agent must reconnect, execute the Refresh() method ¹, and transmit the state snapshot to reconcile any missed transitions during the downtime.

DELIVERABLE B — FLINK ALARM-LIFECYCLE STATE-MANAGEMENT SPECIFICATION

1. Two-tier Responsibility Split

The CAS architecture leverages Apache Flink to implement a sophisticated two-tier stateful processing hierarchy, separating edge autonomy from enterprise aggregation.

- **Edge Flink (Per Site):** A lightweight Flink job operates within the local site's Kafka cluster. Its primary role is first-line deduplication, suppressing repetitive events where the OPC changeMask indicates no meaningful alteration to the ACTIVE_STATE, ACK_STATE, or SUBCONDITION.¹ Furthermore, it executes local chatter and flood detection. This tier guarantees that if the wide-area network severs, local operators accessing a site-specific UI still receive a coherent, functional, and noise-reduced alarm view based on local state.
- **IOC Flink (Enterprise):** This deployment represents the **authoritative source of truth** for the enterprise.¹ It consumes the replicated, normalised streams from all 74 sites. It is responsible for maintaining the definitive ISA-18.2 alarm lifecycle state machine, performing cross-site correlation, calculating complex EEMUA 191 Key Performance Indicators (KPIs), orchestrating operator commands, and sinking the final state to the UI Gateway.¹

2. Job Topology

The IOC Flink environment executes a Unified Alarm Lifecycle Directed Acyclic Graph (DAG). A unified job topology is explicitly mandated over a micro-services architecture. Segmenting state transitions, KPI aggregations, and flood detection into separate Flink jobs would necessitate writing massive volumes of intermediate state back to Kafka, exponentially increasing end-to-end latency and violating the 5–10 second performance target.¹

The DAG proceeds through the following highly parallelised steps:

1. **Kafka Source:** Ingests the cas.edge.telemetry.alarms.normalised topic.
2. **Watermark Assigner:** Generates event-time progress markers.
3. **KeyBy:** Partitions the stream by the unique alarm instance identifier.
4. **ProcessFunction:** Evaluates the ISA-18.2 State Machine, updating internal RocksDB state.

5. **Side Outputs:** Diverts late-arriving events, generates audit trails, and emits calculated KPIs.
6. **Transactional Sink:** Publishes the authoritative state to a log-compacted Kafka topic with exactly-once guarantees.

3. Keying Strategy

Data within the Flink job is deterministically routed and partitioned using a composite string key: `AlarmInstanceKey = concat(siteId, "|", source, "|", conditionName)`.

Rationale: The OPC A&E specification defines a named Condition on a Source as the primary multi-state entity. While an alarm may transition between various subConditions (e.g., a Level condition moving from HI to HIHI), it remains fundamentally the same alarm instance.¹

Therefore, the subCondition is explicitly excluded from the key. This ensures that all state transitions for a specific instrument's alarm condition are processed by the exact same Flink TaskManager, guaranteeing strict ordering and state consistency. The Kafka topic partitions must be perfectly aligned with this key hashing algorithm to avoid costly network shuffles during Flink ingestion.

4. State Primitives

Within the central KeyedProcessFunction, Flink utilises the RocksDB state backend to durably persist the lifecycle of millions of active and historical alarms. The following state primitives are declared:

- **ValueState<AlarmInstanceState>:** The primary data structure holding the current truth of the alarm. It contains the lifecycleState enum, the activeTime timestamp, the ackTime, the currently active subCondition, the normalised severity, and any active shelveExpiry timestamps.
- **MapState<String, Long>:** Utilised to track the historical occurrence frequency of specific sub-conditions for advanced analytical reporting.
- **ListState<Long>:** A temporal list storing the epoch timestamps of the last N transitions. This state is strictly utilised by the Complex Event Processing (CEP) logic to identify rapid chattering.

5. Event-time & Watermarks

To ensure the chronological integrity of the sequence-of-events (SOE), Flink processing relies entirely on Event-Time, eschewing volatile Processing-Time.

- **Timestamp Extraction:** The system extracts the event time directly from the activeTime field (derived from the controller's ftActiveTime).¹
- **Watermark Generation:** A WatermarkStrategy.forBoundedOutOfOrderness is applied, configured with a 10-second delay tolerance. This accommodates inherent network jitter, processing latency at the edge, and minor out-of-order delivery from complex DCS architectures.
- **Idleness:** Given the 74 independent sites, a site experiencing zero alarm activity could theoretically stall the global watermark. The strategy employs

.withIdleness(Duration.ofSeconds(30)) to ensure the watermark continues to advance globally, allowing timers to fire.

- **Low-Fidelity Fallback:** For Foxboro SQL sources flagged with EDGE_INGEST fidelity²³, the system uses the database ingestion timestamp as a proxy for event time. The Flink output explicitly propagates this fidelity flag to the UI, indicating to operators that millisecond SOE precision cannot be guaranteed for these specific assets.¹

6. The Transition Function (ISA-18.2 State Machine)

The core of the Flink application is a KeyedProcessFunction that evaluates incoming events against the current ValueState to enforce the ISA-18.2 state model. The logic parses the Boolean flags derived from the OPC wNewState parameter (ACTIVE, ACKED, ENABLED).¹

State Machine Transition Table:

Current Flink State	Triggering Event (from DCS)	Evaluated OPC Flags	Next Flink State	Output Action
NORMAL	Process exceeds threshold	ACTIVE=true, ACKED=false	UNACKED	Update ValueState, set activeTime. Emit CasStateChange.
UNACKED	Operator acknowledges alarm	ACTIVE=true, ACKED=true	ACKED	Update ValueState, set ackTime. Emit CasStateChange.
UNACKED	Process returns to normal	ACTIVE=false, ACKED=false	RTN-UNACK	LATCH. Condition physically cleared, but requires operator acknowledgment to dismiss.
RTN-UNACK	Operator acknowledges	ACTIVE=false, ACKED=true	NORMAL	Clear activeTime.

	alarm			Emit CasStateChange clearing UI.
ACKED	Process returns to normal	ACTIVE=false, ACKED=true	NORMAL	Clear activeTime. Emit CasStateChange clearing UI.
(Any State)	Instrument placed in maintenance	ENABLED=false	OOS (Out of Service)	Suspend active processing. Emit OOS state to UI.

7. Timers

Flink's robust TimerService is employed to manage complex temporal alarm requirements without blocking processing threads.

- **Event-Time Timers (Debouncing):** Certain mechanical instruments suffer from contact bounce, generating rapid ACTIVE/INACTIVE transitions. Flink registers an event-time timer for an On-Delay or Off-Delay debounce period. If the condition reverts within the 2-second window, the transition is swallowed, suppressing the UI flicker.
- **Processing-Time Timers (Shelving):** When an operator shelves an alarm to temporarily silence it, the system registers a processing-time timer corresponding to the requested duration (e.g., 4 hours). Upon timer expiration, the Flink OnTimer callback executes, automatically forcing the ValueState back to UNACKED (or NORMAL if the physical condition has since cleared).

8. CEP / Flood & Bad-actor Logic

The system integrates continuous pattern matching to identify and mitigate alarm floods and chattering, crucial EEMUA 191 requirements.

- **Chatter Detection:** For every transition, the epoch timestamp is appended to the ListState. If the list size reaches N (e.g., 5 transitions) and the delta between the first and last timestamp is less than T (e.g., 60 seconds), the alarm instance is flagged with a CHATTERING boolean. Flink emits a synthesised meta-alarm alerting the operator to the malfunctioning instrument, and optionally applies a dynamic suppression filter.
- **Flood Detection:** A separate global tumbling window evaluates the aggregate transition rate partitioned by siteld and process area. If the rate exceeds the EEMUA threshold (e.g., >10 alarms per 10 minutes per operator span), a FLOOD_STATE flag is broadcast to the UI

Gateway, commanding the client banner to collapse lower-priority alerts and focus solely on critical events.

9. Exactly-once & State Backend

To guarantee that the HTML5 operator banner never receives duplicate state updates or corrupted histories following a server failure, Flink enforces strict Exactly-Once processing semantics.

- **Checkpointing:** Flink is configured to execute unaligned checkpoints at 30-second intervals. The RocksDB state backend asynchronously flushes incremental snapshots of the in-memory state to a highly durable, on-premise S3-compatible Object Store (or HDFS cluster) suitable for OT deployments.¹
- **Transactional Sink:** The ultimate output to the Kafka topic relies on `DeliveryGuarantee.EXACTLY_ONCE`. Flink orchestrates a two-phase commit (2PC) protocol with the Kafka brokers. Output records remain invisible to downstream consumers (the UI Gateway) until the Flink JobManager successfully confirms the checkpoint barrier, atomically committing the Kafka transaction. This ensures absolute consistency between the internal state and the published output.

10. Recovery & Refresh() Reconciliation

A profound challenge in distributed OT systems occurs when an edge collector crashes, misses several hours of DCS transitions, and subsequently restarts. During this blind spot, a critical alarm may have triggered and then physically cleared. If Flink merely resumes reading, it will never receive the 'clear' event, leaving a ghost alarm permanently stuck in the UNACKED state on the UI.

The CAS architecture solves this via the Refresh() Reconciliation Algorithm:

1. Upon edge agent restart, it connects to the OPC server and executes `Refresh()`.¹
2. The edge publishes a boundary marker event `REFRESH_START`, followed by the snapshot of all currently active/unacked conditions, followed by `REFRESH_END`.
3. As Flink consumes this stream, it buffers the incoming active conditions in a temporary `MapState` scoped to the `siteld`.
4. Upon processing `REFRESH_END`, Flink iterates over its entire durably stored `ValueState` for that specific `siteld`.
5. **Conflict Resolution:** If Flink's internal state indicates an alarm is UNACKED or ACKED, but that specific `AlarmInstanceKey` is **missing** from the buffered Refresh snapshot, it mathematically proves the alarm has physically cleared on the DCS during the outage. Flink forcefully injects a synthetic transition, moving the ghost alarm to RTN-UNACK or NORMAL. This guarantees that the IOC state is perfectly, deterministically reconciled against the DCS ground truth.

11. Backfill Ordering

During the replay of the Kafka S&F buffer after a network partition, events surge into Flink at maximum throughput.

- **Bounded Lateness:** Events delayed by the partition are natively sorted by Flink's Event-Time processing mechanics, provided they fall within the generous watermark tolerance.
- **Extreme Lateness:** If a massive backlog causes events to arrive after the global watermark has irrevocably advanced, Flink intercepts these via a Side Output (LateEventsTag). These profoundly delayed events are not permitted to corrupt the live state machine; instead, they are routed to a secondary Kafka sink dedicated to writing directly to the Historical SQL/PI database²⁴, ensuring regulatory audit trails are preserved without compromising the real-time UI.

12. Serving the Current State

The authoritative, evaluated alarm state must be served to the HTML5 operator clients with sub-second latency.

- **Log Compaction:** Flink publishes the final state to a specially configured Kafka topic: `cas.ioc.state.alarms.current`. This topic is configured for Log Compaction, keyed by the `AlarmInstanceKey`. Kafka automatically deletes older records with the same key, ensuring the topic remains a perfectly sized, perpetually updated snapshot of the current plant reality.
- **UI Gateway:** A highly concurrent middleware service (Node.js/Go) connects as a consumer to this compacted topic. Upon a new browser connection, the gateway rapidly scans the topic from the beginning, building the full table in milliseconds, and streams it to the HTML5 client via Server-Sent Events (SSE) or WebSockets.¹
- **Historical Sink:** A parallel Flink sink writes the immutable transition ledger (the `CasStateChange` records) to the enterprise Historian, PI Server, or relational SQL database to satisfy the "fully scrollable historical overview" requirement.¹

13. Command / Ack Orchestration

Operator interactions originating from the web UI are complex distributed transactions.

1. The operator selects an alarm and clicks "Acknowledge".
2. The UI Gateway publishes a command payload to the `cas.ioc.commands.outbound` topic, keyed by `AlarmInstanceKey`.
3. Flink consumes the command. Crucially, it **does not** transition the core `ValueState` to `ACKED`. Doing so would create an optimistic, unverified state. Flink instead updates an internal flag to `ACK_PENDING`, temporarily disabling the acknowledge button in the UI to prevent duplicate clicks.
4. Flink routes the command to the specific edge topic based on the `siteId`.
5. The Edge Agent receives the command, resolves the volatile `Cookie`, and fires `AckCondition()` to the vendor DCS.¹
6. The DCS processes the acknowledgement and emits a new hardware event with the `OPC_CONDITION_ACKED` flag set to `true`.¹
7. This confirmation event flows up the telemetry pipeline into Flink. Flink evaluates the true hardware confirmation, transitions the state to `ACKED`, and pushes the final state to the

UI. The UI exclusively reflects validated DCS truth.¹

14. KPIs & Analytics

The unified Flink DAG continuously projects the event stream into tumbling analytical windows to calculate ISA-18.2 metrics:

- **Alarm Rate per Operator:** Aggregating UNACKED transitions per 10-minute window, mapped to specific console operator spans.
- **Standing Alarms:** Polling the ValueState to identify conditions remaining in ACTIVE state for > 24 hours.
- **Top Bad Actors:** Executing a TopN calculation over a 1-hour sliding window to rank instruments generating the highest volume of transitions, emitting the dashboard updates periodically.

15. Schemas

The state tracking relies on the heavily governed CasStateRecord schema:

Field Name	Type	Description
alarmInstanceKey	String	The primary routing and compaction key.
currentState	Enum	NORMAL, UNACKED, ACKED, RTN_UNACK, OOS, SHELVED.
activeTime	Long	Preserved SOE timestamp of activation.
ackTime	Long	Timestamp of validated operator action.
currentSeverity	Int	Normalised severity driving UI colour-coding.
isChattering	Bool	Flag indicating the CEP engine has detected noise.
lastUpdateTime	Long	Event-time of the most recent transition.

16. Scaling, HA & Sizing Inputs

- **Parallelism Alignment:** The Flink job parallelism is configured to map precisely to the Kafka partition count of the inbound telemetry topics (e.g., 24 or 48 partitions), ensuring optimal thread distribution and avoiding data skew.
- **Key Cardinality:** Given 74 sites, the architecture is sized assuming an upper bound of 150,000 distinct configured alarm tags per site, resulting in a global RocksDB key cardinality of over 11 million instances. RocksDB easily scales to accommodate this on standard NVMe SSDs.
- **Sizing Inputs Required: OPEN ITEM:** PDO must provide the estimated concurrent user load (e.g., 50 vs. 500 active browser sessions) to accurately scale the Node.js/Go UI Gateway horizontally.¹ **OPEN ITEM:** PDO must define the definitive online historical retention policy (e.g., 1 year vs. 7 years) to appropriately dimension the enterprise SQL/PI storage arrays.¹ **OPEN ITEM:** The requirement for AVEVA System Platform integration remains "Scope-TBC". If activated, Flink must be configured with an additional OPC UA or MQTT sink to push the consolidated state to the Wonderware infrastructure, affecting cluster CPU sizing.¹

17. Test & Verification

The Flink engine undergoes comprehensive validation:

- **State Machine Verification:** Employing Flink's KeyedOnInputStreamOperatorTestHarness, engineers inject arrays of synthetic OPC state permutations (including illegal transitions) to assert that the ValueState perfectly adheres to the ISA-18.2 matrix.
- **Reconciliation Simulation:** Unit tests simulate network outages, injecting REFRESH boundaries containing subsets of active alarms, validating that the Flink logic correctly identifies and forcefully clears the simulated ghost alarms.
- **Chaos Engineering:** During staging, Kafka Testcontainers are used to programmatically kill Flink TaskManagers mid-processing during an induced alarm flood. The verification suite queries the compacted output topic to guarantee that, post-recovery, zero duplicate state records exist, mathematically proving the exactly-once transactional semantics.

Works cited

1. YG260519-PDO CAS Clarification Register.xlsx
2. Try the Knowledge Base < - Honeywell Process Automation, accessed May 23, 2026,
https://process.honeywell.com/content/dam/process/en/documents/document-lists/doc_list_bg/2022/Honeywell_Benefit_Guardian_Newsletter-Q1%202022.pdf
3. Help connecting 3rd party equipment to DeltaV OPC- Already tried OPC remote but didn't work - Emerson Exchange 365, accessed May 23, 2026,

- <https://emersonexchange365.com/community-hubs/deltav-community-connect/f/deltav-discussions-questions/7824/help-connecting-3rd-party-equipment-to-deltav-opc--already-tried-opc-remote-but-didn-t-work>
4. Open Interfaces - Yokogawa, accessed May 23, 2026,
<https://web-material3.yokogawa.com/IM32Q05B10-31E.pdf>
 5. Exaopc Setup for Exaquantum Guide | PDF | Server (Computing) - Scribd, accessed May 23, 2026,
<https://www.scribd.com/document/807421664/Exaquantum-Setup-and-Troubleshooting-Guide>
 6. System 800xA 5.1 Configuration - ABB, accessed May 23, 2026,
https://library.e.abb.com/public/0f6a2957da78e928c1257b400026fd70/3BDS011222-510_F_en_System_800xA_5.1_Configuration.pdf
 7. System 800xA 6.0 Configuration - ABB, accessed May 23, 2026,
https://library.e.abb.com/public/40dfb2533af14dbe8d7b4d3e8c3d2c87/3BDS011222-600_C_en_System_800xA_6.0_Configuration.pdf
 8. DeltaV Client Application connection problem - Emerson Exchange 365, accessed May 23, 2026,
<https://emersonexchange365.com/products/control-safety-systems/f/deltav-discussions-questions/4560/deltav-client-application-connection-problem>
 9. PI Interface for OPC Alarms and Events - OSIsoft Partner Portal, accessed May 23, 2026,
<https://osisoft.my.site.com/PISquare/s/topic/0TO1I000000OPwkWAG/pi-interface-for-opc-alarms-and-events>
 10. Schneider Electric / Invensys / Foxboro I/A Series Software Manual | ArtisanTG, accessed May 23, 2026,
https://www.artisanTG.com/info/Invensys_Foxboro_IA_Series_Software_Manual.pdf
 11. [PSS 21S-6C1 B3] AIM*Historian Software - Schneider Electric, accessed May 23, 2026,
<https://paresource.schneider-electric.com/iaseries/pss/21s6/21s6c1b3.pdf>
 12. [PSS 41S-4INBATCH.D] EcoStruxure Foxboro DCS Batch Management Software - Schneider Electric, accessed May 23, 2026,
<https://paresource.schneider-electric.com/iaseries/pss/41s4/41s4inbatch.pdf>
 13. Experion-HS-Application-Development-Guide, accessed May 23, 2026,
<https://www.dgfg.nl/wp-content/uploads/2018/02/Experion-HS-Application-Development-Guide.pdf>
 14. iFIX OPC A&E Server Setup Guide | PDF | Component Object Model | Computing - Scribd, accessed May 23, 2026,
<https://www.scribd.com/document/858619596/OPC-A-and-E-Server-for-iFIX>
 15. Honeywell DCS Configuration Guide | PDF | Computer Network | Login - Scribd, accessed May 23, 2026,
<https://www.scribd.com/document/916018147/Honeywell-DCS-EPKS-Getting-Started-part1B>
 16. DeltaV Configuration PDF | PDF | Scada | Systems Engineering - Scribd, accessed May 23, 2026,
<https://www.scribd.com/document/483809796/316166572-DeltaV-Configuration->

[pdf](#)

17. AMS Machine Works v2.1 User Guide | Emerson Electric, accessed May 23, 2026, <https://www.emerson.com/is/content/emerson/en/systems-and-software/ams/manuals-and-guides/documents/ams-machine-works-v2-1-user-guide.pdf>
18. OPC AE Specification | PDF | Component Object Model | Server (Computing) - Scribd, accessed May 23, 2026, <https://www.scribd.com/document/522913894/OPC-AE-Specification>
19. AAM Manual Builder 2ed | PDF | Windows Registry | Databases - Scribd, accessed May 23, 2026, <https://www.scribd.com/document/397862378/AAM-Manual-Builder-2Ed>
20. 800xA System, Engineering Planning and Concepts - ABB, accessed May 23, 2026, https://library.e.abb.com/public/fc7813f0a7c647599e33e9e6c7b930b2/3BSE041389-600_B_en_System_800xA_6.0_System_Planning.pdf?x-sign=ysc4xfE7agg1yIW5AdDVq1xYye16tNOK7cfD%2FC3jRUH5XneaBgd9hgjLJPEbfKBf
21. GEH-6862 ToolboxST User Guide for Mark VIeS Functional Safety Systems PDF document, accessed May 23, 2026, <https://device.report/m/a6a2eeadcf38049e93b8642032e64a2db90ba242b9f56134d004fd7d419ec4ce>
22. Experion Server Specifications and PDF - Scribd, accessed May 23, 2026, <https://www.scribd.com/document/398529611/Experion-Server-Specifications-and-pdf>
23. Foxboro Batch Interface to the PI System, accessed May 23, 2026, https://cdn.osisoft.com/interfaces/953/PI_FoxBatch_1.0.0.9.doc
24. Associate Asset with Alarms and Events using data from Alarms and Events Server (OPC A&E) - OSIsoft Partner Portal, accessed May 23, 2026, <https://osisoft.my.site.com/PISquare/s/question/0D51I00004UHKw5SAD/associate-asset-with-alarms-and-events-using-data-from-alarms-and-events-server-opc-ae>